National School of Arts and Sciences
COMP1004 – Database Systems
Semester 2 (S2), 2026
Fremantle, Sydney Campus
Assessment 2 Brief

Module Code:	COMP1004
Module Title:	Database Systems

Assessment Number:	2
Assessment Weight:	40% (part1- 30% & Part2- 10%)
Assessment Individual/Group:	Individual
Assessment Type:	A Complete Database with project report and Project demonstration
Assessment Time/Word Count Restrictions:	1 SQLite database file 1500 - 2500 words in Cadmus report
Assessment Time/Word Count Limit Consequences:	You are encouraged to stay within the specified word count. However, if you need additional words to clearly explain your work, you may go slightly beyond the limit. Your detailed explanation will be read and appreciated, though we recommend being as concise and focused as possible.

Issue Date:	24/08/2026
Submission Date:	3/10/2026
Planned Feedback Date:	Within 4-5 working weeks
Mode of Submission:	Online via Blackboard
Number of copies to be submitted:	• A report in Cadmus • One complete project directory containing ass necessary files and folders
Blackboard submission:	• Cadmus assignment link: Students will directly write their report on Cadmus. • Project submission link: The **zip file** of the complete project directory should be submitted through this link. Your course lecturer will demonstrate this process during class.

National School of Arts and Sciences
COMP1004 – Database Systems
Semester 2 (S2), 2026
Fremantle, Sydney Campus
Assessment 2 Brief

Table of Contents

Table of Contents

Introduction: 3
Problem Scenario: 3
System Overview: 3
Users: 3
Order management: 4
Payment management: 4
Current data management Problems: 4
Database Requirements: 5
System requirements: 5
Functional Requirements: 5
Non-Functional Requirements: 6
Task requirements (Part 1 of AT1): 6
Task 1: Requirement Analysis & Attribute Identification 6
Task 2: Master Table Creation (UNF) 6
Task 3: Normalization 7
Task 4: Relational Schema Design 7
Task 5: SQLite Database Implementation 7
Task 6: SQL queries, Views, and Indexing 7
A. Basic Queries: 7
B. Join Queries: 7
C. Aggregate Queries: 8
D. SQL Views: 8
E. Indexing: 8
Task 7: Project Report 8
Evidence Requirements: 9
Environment & Tools: 9
Submission Instructions: 9
Part 2: Project Defense and Demonstration 10
Assessment Criteria and Rubric for marking 11
Learning Outcomes 13

National School of Arts and Sciences
COMP1004 – Database Systems
Semester 2 (S2), 2026
Fremantle, Sydney Campus
Assessment 2 Brief

Introduction:

The **UNDA Student Services Online Ordering and Membership System (SSOMS)** is designed to support the University of Notre Dame Australia's student and staff services through an online ordering platform.

The system will allow students and staff to purchase university-related products and services, including textbooks, stationery, study materials, university merchandise, gym-related products and gym memberships.

For this assessment, students will design and implement a **relational database using SQLite**. The assessment focuses on requirement analysis, ER modelling, normalization, relational database design, SQL queries, constraints, views and basic indexing.

Problem Scenario:

System Overview:

The University of Notre Dame Australia (UNDA) Student Services is planning to introduce an **Online Ordering and Membership System (SSOMS)** to provide students and staff with a convenient platform for purchasing products and services related to university study, student life and recreation.

The system will allow students and staff to browse available products, place orders, and make payments.

Products and Categories:

The system will offer a range of university-related products, including:

- Textbooks and reference books
- Stationery
- Study materials
- University merchandise
- Sports clothing
- Gym-related products

Products will have information such as a product name, description, price, category and available stock quantity. Products will be classified into appropriate categories to help users and Student Services staff manage and search for products.

Users:

The system will be used by both **students and staff**.

The system should maintain relevant information about each user, such as their name, email address, phone number and user type. Users should be able to place orders and, where eligible, purchase gym memberships.

Order management:

Users will be able to select products and place orders through the system.

For each order, the system should maintain information such as the order date, user, products purchased, quantities, prices and order status.

An order may contain **multiple products**, and the same product may be included in **many different orders**.

The system should support order statuses such as:

Pending, Confirmed, Processing, Completed and Cancelled.

Payment management:

The system should maintain information about payments associated with orders and gym memberships.

Payment information may include:

- Payment date
- Payment amount
- Payment method
- Payment status

Possible payment statuses include **Pending, Successful, Failed and Refunded**.

Students must **not store real credit-card numbers, passwords or other sensitive payment information** in their database.

Current data management Problems:

At present, information about users, products, orders, payments and memberships may be maintained through separate records or manual processes.

This can result in:

- Duplicate information
- Inconsistent records
- Difficulty tracking product stock
- Difficulty tracking orders
- Difficulty identifying active or expired memberships

- Difficulty generating useful reports

Student Services therefore requires an integrated and well-structured relational database.

Database Requirements:

The database should allow Student Services to store and manage information about users, products, categories, orders, and payments.

The system should also support common operations such as:

- Identifying available products
- Finding pending orders
- Calculating total sales
- Identifying popular products
- Viewing customer order information
- Viewing products by category
- Identifying active or expired gym memberships
- Calculating membership sales

Students are required to **analyse the scenario, identify the relevant entities, attributes and relationships, and design and implement an appropriate relational database using SQLite**.

The database should be designed to maintain data integrity, reduce unnecessary data redundancy and support efficient retrieval of information using SQL.

System requirements:

Functional Requirements:

The SSOMS must support the following functionalities:

- User Management – Store information about students and staff, including an appropriate identifier, name, email, phone and user type. Student-specific and staff-specific information may also be stored where appropriate.
- Product and Service Management – Store information about textbooks, reference books, stationery, study materials, merchandise, and gym-related products.
- Category Management – Classify physical products into appropriate categories.
- Order Management – Allow users to place orders containing one or more products and/or eligible services, and maintain order date, status, quantities and prices.
- Payment Management – Record payment amount, payment date, payment method and payment status. Students must not store real credit-card numbers or other sensitive payment credentials.

- Reporting – Support reporting such as total orders, total sales, sales by category, popular products, low-stock products, number of gym memberships purchased and popular membership plans.
- Data Retrieval – Users must be able to retrieve information about products, orders, payments and memberships using SQL queries.

Non-Functional Requirements:

- Database Integrity – The SQLite database must use appropriate primary keys, foreign keys and constraints such as NOT NULL, UNIQUE, DEFAULT and CHECK.
- Data Consistency – Transactional data should be represented consistently and accurately.
- Usability – Database structures, queries and scripts should be clearly organized and understandable.
- Scalability – The design should support additional users, products, and orders without requiring fundamental changes to the database structure.
- Portability – The SQLite implementation must be runnable using standard database software available to students.
- Performance – The database should provide efficient retrieval of commonly requested information. Appropriate indexes should be considered where necessary.

Task requirements (Part 1 of AT1):

Based on the system requirements and problem scenario, students are expected to complete the following tasks:

Task 1: Requirement Analysis & Attribute Identification

- Identify all relevant entities involved in the system, such as User, Product and all.
- Identify attributes for each entity, including primary identifiers and descriptive attributes.
- Identify candidate keys and appropriate relationships.
- Identify relationship cardinalities.
- Design an initial Entity–Relationship (ER) model based on the analysis.

Task 2: Master Table Creation (UNF)

- Create one large unnormalized master table containing the information required by the scenario.
- The table should demonstrate realistic redundancy and repeating groups.
- Ensure that the attributes required to support ordering, products, and payments are represented.

- Use the UNF table as the starting point for normalization.

Task 3: Normalization

- Normalize the master table through 1NF, 2NF, and 3NF.
- Clearly identify functional dependencies, partial dependencies and transitive dependencies.
- Explain how normalization reduces redundancy and update, insertion and deletion anomalies.

Task 4: Relational Schema Design

- Design the final relational schema based on the normalized structure.
- Provide table names and attributes.
- Identify primary keys, candidate keys and foreign keys.
- Identify appropriate relationship cardinalities.
- Create a complete database schema using Use DBML tool (dbml.dbdiagram.io).

Task 5: SQLite Database Implementation

- Implement the relational schema in SQLite Studio or an approved SQLite environment.
- Apply PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK constraints where appropriate.
- Populate the database with sufficient fictitious sample data to demonstrate all required queries.
- Minimum recommended data: 10 users, 4 categories, 10 physical products, 10 orders, 15 order items, and 8 payments.
- Ensure that the data supports all SQL queries, views, and indexing tasks.

Task 6: SQL queries, Views, and Indexing

Students must write and test the following SQL queries:

A. Basic Queries:

- Display all students' details.
- Display products with price above a specified value (For example price above $100).
- Display all orders placed in the month of July 2026.
- Display all payment details where the payment method is '**credit card**'.

B. Join Queries:

- Display order details ordered by all staff.
- Display payment information together with order and user information.
- Display products and their categories.
- Display the student's name, order ID, and payment method for all completed orders.

C. Aggregate Queries:

- Calculate total sales amount for each product.
- Calculate the total number of orders placed by each student.
- Find users whose total spending is above the average user spending.
- Display the staff who have placed the highest number of orders.

D. SQL Views:

- Create two meaningful views in SQLite that provide useful information for the student Services system.
- Justify the purpose and usefulness of each view, explaining why the selected information is relevant to the system. (In the Cadmus report along with the screenshots, you need to address this point.)

E. Indexing:

- Identify columns frequently used in WHERE, JOIN, ORDER BY or GROUP BY clauses.
- Create at least one in SQLite.
- Provide SQL statements for the indexes.
- Justify why the index may improve query performance. (In the Cadmus report along with the screenshots, you need to address this point.)

Note: You may either save all your queries in your database or provide them in a separate Word/text file and submit the file along with project directory. Your lecturer must have access to all queries required to taste and verify your database implementation. Failure to provide the required queries will result in a deduction of 3 marks.

Task 7: Project Report

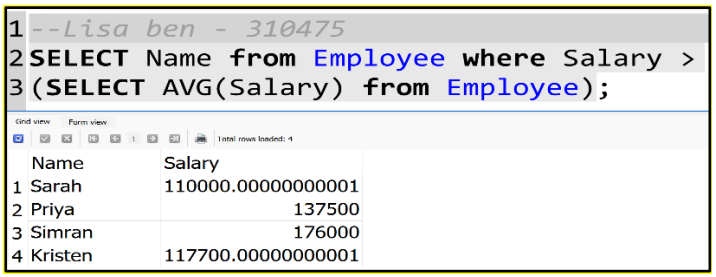
Prepare a **project report (1500 - 2500 words)** including:

- Introduction, problem scenario and objectives.
- Task 1: Requirement analysis, entities, attributes and ER diagram.
- Task 2: UNF master table.
- Task 3: Step-by-step normalization to BCNF.
- Task 4: Relational schema including keys and relationships.
- Task 5: SQLite database implementation, including table creation and sample data.
- Task 6: SQL DDL/DML queries, query results, views, and indexes.
- A short reflection on challenges and lessons learned.

Note: In the Cadmus project report, do not write SQL queries. Instead, provide screenshots along with each question. The screenshots must clearly show both the SQL query and its output/result. Please refer to the example under **'Evidence Requirements'**.

Evidence Requirements:

- The SQLite database file must be named: SSOMS_studentName_studentID.db. For example: SSOMS_John_1234567.db
- Students must include their own fictitious/academic user record in the database, where appropriate, without including sensitive information. For example: Name, ID, email address.
- Every SQL screenshot must show the SQL query and its output.
- SQL query screenshots should include a comment in the format: -- **Your name, Student number**. For example:



```
1 --Lisa ben - 310475
2 SELECT Name from Employee where Salary >
3 (SELECT AVG(Salary) from Employee);
```

	Name	Salary
1	Sarah	110000.00000000001
2	Priya	137500
3	Simran	176000
4	Kristen	117700.00000000001

- All SQL scripts used for the assessment must be submitted with the project.

Environment & Tools:

You are required to use the following tools:

- SQLite Studio / SQLite
- Draw.io for ER diagram
- dbml.dbdiagram.io for relational database schema

Submission Instructions:

The assessment must be completed individually. You must not share, in part or whole, your assessment with another party other than the module convenor and for the purpose of submission to the university. You must ensure that the University's academic misconduct guidelines are followed in their entirety.

- The assessment must be completed individually.
- Students will directly write their report on Cadmus.
- All screenshots and diagrams (ER and database schema) must be included in the Cadmus project report.
- Submit a ZIP project file through the designated Blackboard submission link.
- The ZIP file should contain the SQLite database file, SQL script/query file, and any other required project files.
- Students must ensure that all work complies with the University's academic misconduct and academic integrity requirements.

Part 2: Project Defense and Demonstration

- Submit your **Part 1** work by **3 October 2026 (end of Week 10)**.
- From **Week 11**, you will be required to **defend and demonstrate your project** during your tutorial session.
- Your lecturer may ask technical questions about your project and may ask you to write or execute queries using **SQLite** based on your submitted work.
- Your marks for **Part 1 and Part 2** will be based on your overall performance.
- You must be able to explain and demonstrate your own work. If you are unable to adequately defend your project, your **Part 1 marks may be affected**.

National School of Arts and Sciences
COMP1004 – Database Systems
Semester 2 (S2), 2026
Fremantle, Sydney Campus
Assessment 2 Brief

Assessment Criteria and Rubric for marking

Your assessment will be graded according to the following criteria:

Grading criteria	Technical Competence	Problem Solving	Communication Skills
Mark band			
80-100	Displays exceptional skills with comprehensive implementation and design considerations.	Exceptional problem-solving demonstrated in advanced implementation and design.	Exceptional communication skills demonstrated through a comprehensive and articulate report.
70-79	Exhibits advanced skills with comprehensive software development, and in-depth design considerations.	Advanced problem-solving skills apparent in advanced software design and development.	Advanced communication in explaining complex software development concepts.
60-69	Good, consistent knowledge and understanding of the material, main concepts, key theories, and professional practice at this level. Exhibits advanced skills in application development.	Strong problem-solving skills in problem solving.	Strong communication in explaining complex programming and problem solving concepts.
50-59	Sound, routine knowledge and understanding of the material, main concepts and key theories. Some flaws may be evident. Shows improved skills with refined implementation.	Improved problem-solving seen in refining solution.	Improved communication skills seen in summarising design and implementation.

40-49	Demonstrates basic skills in developing a functional application	Basic problem-solving evident in development tasks.	Basic communication evident in summary of implemented solution
30-39	Displays limited proficiency, with significant deficiencies in the programming.	Demonstrates weak problem- solving abilities, struggling with basic tasks.	Demonstrates deficient communication skills, struggling to articulate ideas and concepts.
0-29	Demonstrates severely inadequate skills, unable to create a basic functional program.	Lacks any evident problem solving ability, unable to address basic tasks.	Poor communication skills, unable to provide clear summary.

National School of Arts and Sciences
COMP1004 – Database Systems
Semester 2 (S2), 2026
Fremantle, Sydney Campus
Assessment 2 Brief

Learning Outcomes

This assessment will enable students to demonstrate the following learning outcomes as stated in the module outline:

- Evaluate critically the performance, security, and privacy aspects associated with database systems and the ethical considerations in handling sensitive data.
- Proficiently construct and execute complex SQL queries, and subqueries, to retrieve, manipulate, and manage data effectively within a database.
- Design and implement a fully functional database system for real-world applications, utilising both SQL and No-SQL database technologies, and demonstrating an understanding of data modelling, CRUD operations, and the integration of hybrid solutions.
- Efficiently execute database projects with strong problem-solving, information management, numeracy, and a dedication to professional growth. Stay current on research trends, industry advancements, and ethical aspects in database management.